

The Ackermann Function and the Turing Machine: The Exact Boundary of Primitive Recursion

For readers working in formal languages, mathematical logic, and the foundations of computability.

Abstract

The Ackermann function is the canonical witness to the strict inclusion

$$\{\text{primitive recursive functions}\} \subsetneq \{\text{total Turing-computable functions}\}.$$

It is total and effectively computable — a Turing machine evaluates it on every input and halts — yet it is not primitive recursive. We make this precise in three equivalent guises: (i) in the *function* sense, the values $A(m, n)$ escape every primitive-recursive growth bound; (ii) in the *resource* sense, **no Turing machine computing A has a primitive-recursive bound on its running time**; and (iii) in the *proof-theoretic* sense, A is provably total in Peano Arithmetic but sits above the fragment that proves the totality of all primitive-recursive functions. The mechanism is the **fast-growing hierarchy** $(F_m)_{m \in \mathbb{N}}$: A is a single *level-shift* away from F_m , while every primitive-recursive function is dominated by some *fixed* level F_m . The diagonal $n \mapsto A(n, n)$ therefore climbs the hierarchy without bound and cannot be primitive recursive. We give full proofs of the domination lemma and the non-primitive-recursiveness of A , and situate the result in the theory of provable totality (ordinal ε_0) and the Kirby-Paris hydra interpretation.

1. Thesis and roadmap

A reader might ask: *in what sense* does the Ackermann function “relate to” a Turing machine? The naive answer — “a Turing machine can compute it” — is true but vacuous, since every total computable function is Turing-computable by definition. The substantive relationship is that A is the **minimal, canonical boundary object** between two classes that a Turing machine straddles:

$$\underbrace{\text{PR}}_{\text{provably PR-bounded computation}} \subsetneq \underbrace{\text{total computable}}_{\text{halts, but no PR time bound}} = \text{Turing-computable} = \mu\text{-recursive}.$$

Concretely we establish:

1. **A is Turing-computable.** Its recursive definition is effective and terminates; we exhibit the machine and the μ -recursive construction (§3).
2. **A is not primitive recursive** (§6). The proof uses the fast-growing hierarchy and a domination lemma: every PR function is bounded by some fixed level F_m , whereas the diagonal of A outruns every fixed level.
3. **The resource reading.** f is PR iff some Turing machine computes f within a PR time bound (§4). Hence A being non-PR is *equivalent* to: every Turing machine computing A runs for more than any PR function of its input. A is the first total function a Turing machine can compute that no PR bound can certify.
4. **The proof-theoretic reading.** A is provably total in PA; the provably-total functions of PA are exactly those dominated by the fast-growing hierarchy up to ordinal ε_0 (§7). A 's growth is precisely a finite level of that hierarchy.

We fix notation in §2, develop the fast-growing hierarchy in §5, and close with the hydra interpretation and open directions.

2. The Ackermann-Péter function

We use the two-argument (Péter) form, the standard one in the literature on primitive recursion:

$$\begin{aligned} A(0, n) &= n + 1, \\ A(m + 1, 0) &= A(m, 1), \\ A(m + 1, n + 1) &= A(m, A(m + 1, n)). \end{aligned} \tag{1}$$

(*Historical note.* Ackermann (1928) introduced a related three-argument function in his study of a gap in Hilbert's finitist programme; Péter (1956–58) recast it as (1) and proved it is not primitive recursive. The name “Ackermann function” now refers to (1).)

2.1 The first rows: one iteration level per row

Unfolding (1) gives closed forms for the low rows. Each row is exactly one *iteration level* above the previous:

m	$A(m, n)$	operation
0	$n + 1$	successor
1	$n + 2$	addition (from 2)
2	$2n + 3$	multiplication
3	$2^{n+3} - 3$	exponentiation
4	$2 \uparrow\uparrow (n + 3) - 3$	tetration (power tower)

(2)

where $\uparrow\uparrow$ is Knuth's double arrow: $a \uparrow\uparrow 1 = a$ and $a \uparrow\uparrow (k + 1) = a^{a \uparrow\uparrow k}$.

Proof sketch (by induction on m). $m = 0$ is immediate. Suppose $A(m, n) = 2n + 3$. Then $A(m + 1, 0) = A(m, 1) = 2 \cdot 1 + 3 = 5 = 2^{1+3} - 3$, and $A(m + 1, n + 1) = A(m, A(m + 1, n)) = 2A(m + 1, n) + 3 = 2(2^{n+3} - 3) + 3 = 2^{n+4} - 3$, so $A(m + 1, n) = 2^{n+3} - 3$. The $m = 3 \rightarrow 4$ step is identical in shape but replaces the linear recurrence $x \mapsto 2x + 3$ by $x \mapsto 2^x - 3$, yielding a power tower: $A(4, n + 1) = 2^{A(4, n)} - 3$ with $A(4, 0) = 13 = 2 \uparrow\uparrow 3 - 3$, hence $A(4, n) = 2 \uparrow\uparrow (n + 3) - 3$. $\square$

The pattern is the intuitive engine of everything that follows: **each increment of the first argument adds one level of iteration** (successor $\rightarrow + \rightarrow \times \rightarrow \rightarrow \uparrow\uparrow$), which is precisely what the fast-growing hierarchy formalizes.

The growth is not merely fast — it is *unwritable* within a few rows. Verified by direct computation:

- $A(4, 1) = 65\,533$;
- $A(4, 2) = 2 \uparrow\uparrow 5 - 3 = 2^{65\,536} - 3$, an integer with **19 729** decimal digits;
- the diagonal value $a(4) = A(4, 4) = 2 \uparrow\uparrow 7 - 3$ is a power tower of seven 2's; its *decimal digit count* is itself the 19 728-digit integer $\approx 6.03 \times 10^{19\,727}$.

2.2 Totality

A is **total**: $A(m, n)$ is defined for all $m, n \in \mathbb{N}$.

Proof (by induction on m). $A(0, n) = n + 1$ is total. Assume $A(m, \cdot)$ is total. We show $A(m + 1, \cdot)$ is total by induction on n : $A(m + 1, 0) = A(m, 1)$ is defined by the inductive hypothesis on m ; and $A(m + 1, n + 1) = A(m, A(m + 1, n))$ is defined because $A(m + 1, n)$ is (inner induction) and $A(m, \cdot)$ is total (outer induction). $\square$

This is a genuine, constructive termination argument — not an appeal to a completeness theorem — and it is what licenses the Turing machine of §3.

3. A is Turing-computable

3.1 The machine

We describe a deterministic one-tape Turing machine M_A that, on input the pair (m, n) (unary or binary), halts with $A(m, n)$. The tape plays two roles: a **stack of pending frames** (m', n') and a **work area** for the current value.

M_A pushes the frame (m, n) and loops:

1. Pop the top frame (m', n') .
2. If $m' = 0$: the value is $n' + 1$; deliver it (to the continuation frame below, or as the final output).
3. If $n' = 0$: push $(m' - 1, 1)$ [the clause $A(m + 1, 0) = A(m, 1)$].
4. If $n' > 0$: push a *continuation* frame $(m' - 1, \cdot)$ and then push $(m', n' - 1)$. The machine first computes $A(m', n' - 1)$; when that value v is returned, it pushes $(m' - 1, v)$, thereby computing $A(m' - 1, v) = A(m', n')$ [the clause $A(m + 1, n + 1) = A(m, A(m + 1, n))$].

Because A is total (§2.2), the stack is always eventually exhausted, so M_A halts on every input. Finite control plus a tape-as-stack is a legitimate Turing machine, so A is **Turing-computable**.

3.2 The μ -recursive reading

Equivalently, A is a **general (μ -)recursive function**. The recursion (1) is *not* a primitive recursion — in the clause $A(m + 1, n + 1) = A(m, A(m + 1, n))$ the argument of the outer call is itself the value of a recursive sub-computation, so the number of “steps” is not bounded by a primitive-recursive function of (m, n) . It is, however, a *total* recursive definition, and by the fundamental equivalences of the 1930s —

$$\text{general recursive} = \mu\text{-recursive} = \text{Turing-computable} = \lambda\text{-definable}$$

(Kleene 1936; Church 1936; Turing 1936) — A is computed by a Turing machine. The point of §6 is that A lies in this class **but not** in the primitive-recursive sub-fragment.

4. The primitive-recursive functions, and their exact Turing-machine characterization

A function is **primitive recursive (PR)** iff it belongs to the smallest class containing the zero function, the successor $S(x) = x + 1$, and the projections $\pi_i^k(x_1, \dots, x_k) = x_i$, and closed under **composition** and **primitive recursion**:

$$f(x, 0) = g(x), \quad f(x, t + 1) = h(x, t, f(x, t)) \implies f \text{ is PR if } g, h \text{ are.}$$

The key to the whole essay is the following bridge between the syntactic class “PR” and the operational class “Turing machine with a PR clock”.

Theorem 1 (PR = PR-bounded-time Turing-computable). A function $f : \mathbb{N}^k \rightarrow \mathbb{N}$ is primitive recursive iff there exist a deterministic one-tape Turing machine M computing f and a primitive-recursive function $t : \mathbb{N} \rightarrow \mathbb{N}$ such that M halts within $t(|x|)$ steps on every input x .

Proof.

($\Rightarrow$) By structural induction on f . The initial functions are computed by trivial machines with obvious (linear, hence PR) step bounds. Composition: run the machines for the sub-functions in sequence; the step bound is the sum (a PR function) of the sub-bounds. Primitive recursion: $f(x, t)$ is computed by a machine that simulates the step h exactly t times; since t is part of the input and h has a PR step bound, the total bound is PR (a PR-bounded loop). Every PR function is thus computed by a machine whose running time is PR-bounded.

($\Rightarrow$ **reverse**) Conversely, suppose M computes f and halts within $t(|x|)$ steps with t PR. Make the halting states *absorbing* (a config in a halting state maps to itself). The configuration of M after s steps is a function cfg_s of the start configuration; since the transition relation of a one-tape TM is finite (hence PR, indeed Δ_0), the map $\text{cfg} \mapsto \text{cfg}'$ is PR, and therefore its s -fold iterate $\text{cfg}_s = T^{\circ s}(\text{cfg}_0)$ is PR in (cfg_0, s) — **bounded iteration of a PR function is PR**. Taking $s = t(|x|)$ (a PR value) and reading the output from $\text{cfg}_{t(|x|)}$ (which equals the halting configuration, since M halted by then) yields f as a PR function. $\square$

Corollary 1. f is *not* primitive recursive iff **no** Turing machine computing f has a primitive-recursive time bound.

This corollary is the resource-theoretic translation of everything that follows: once we prove A is not PR, it is automatic that every Turing machine computing A (including M_A of §3) runs for more than any PR function of its input.

5. The fast-growing hierarchy

Define the (finite-level) **fast-growing hierarchy** $(F_m)_{m \in \mathbb{N}}$ by

$$F_0(n) = n + 1, \quad F_{m+1}(n) = F_m^{\circ n}(n), \quad (3)$$

i.e. $F_{m+1}(n)$ is the n -fold iterate of F_m , evaluated at n . The first levels are

$$F_0(n) = n+1, \quad F_1(n) = 2n, \quad F_2(n) = 2n^2, \quad F_3(n) \approx 2^n n^{2^n}, \quad F_4(n) \approx \text{tower of height } n.$$

Two facts about (F_m) do all the work.

Lemma 1 (PR-closure). The function $\Phi(m, n) = F_m(n)$ is primitive recursive.

Proof. By induction on m . $\Phi(0, n) = n+1$ is PR. For the step, $\Phi(m+1, n) = F_m^{\circ n}(n)$: define $I(m, n, s)$ by $I(m, n, 0) = n$ and $I(m, n, s+1) = F_m(I(m, n, s)) = \Phi(m, I(m, n, s))$. The body is PR by the inductive hypothesis, so I is PR by primitive recursion in s , and $\Phi(m+1, n) = I(m+1, n, n)$ is PR by composition. $\square$

Lemma 2 (Domination / exhaustiveness). Every primitive-recursive $f : \mathbb{N}^k \rightarrow \mathbb{N}$ is dominated by a single level of the hierarchy: there exist $m \in \mathbb{N}$ and N such that

$$f(x_1, \dots, x_k) \leq F_m(\sum_i x_i) \quad \text{for all } x \text{ with } \sum_i x_i \geq N.$$

Proof (structural induction on f , maintaining that the size function is the sum $\langle x \rangle = \sum_i x_i$).

- **Successor.** $S(x) = x+1 \leq F_0(\langle x \rangle)$ for $k = 1$; in general S applied to one coordinate is $\leq F_1(\langle x \rangle)$ eventually.

- **Projections.** $\pi_i(x) = x_i \leq \langle x \rangle \leq F_0(\langle x \rangle) + 1 \leq F_1(\langle x \rangle)$ eventually.
- **Constants.** $c \leq F_1(\langle x \rangle) = 2\langle x \rangle$ for $\langle x \rangle \geq c/2$ (eventual).
- **Composition.** Let $f = h(f_1, \dots, f_r)$. By induction $f_i(x) \leq F_{m_i}(\langle x \rangle)$ and $h(y) \leq F_M(\langle y \rangle)$, eventually. Then

$$f(x) \leq F_M(\sum_i f_i(x)) \leq F_M(r \cdot F_{\max m_i}(\langle x \rangle)) \leq F_{M+\max m_i+2}(\langle x \rangle),$$

using monotonicity and the standard **level-additivity** of the hierarchy, $F_p(F_q(n)) \leq F_{p+q+1}(n)$ (an n -fold iterate of F_q from a value already $\geq n$ reaches level p in at most $p + q + 1$ overall levels). The argument stays $\langle x \rangle$; only the level index grows.

- **Primitive recursion.** Let $f(x, 0) = g(x)$, $f(x, t+1) = h(x, t, f(x, t))$, with g, h PR. By induction $g(x) \leq F_a(\langle x \rangle)$ and $h(x, t, z) \leq F_b(\langle x \rangle + t + z)$ eventually. The recursion is exactly a *bounded iteration* of the step h : by induction on t one obtains

$$f(x, t) \leq F_{b+1}(\langle x \rangle + t) \quad \text{eventually in } t,$$

because F_{b+1} is defined as the iterate of F_b , and a primitive recursion of depth t is t iterations. This is the heart of the lemma: **one level of primitive recursion consumes exactly one level of the fast-growing hierarchy.** $\square$

Lemma 2 is the content: the fast-growing hierarchy is a *scale that exhausts* the primitive-recursive functions. No matter how a PR function is built — however many nested primitive recursions — it is bounded by one *fixed* level F_m .

Lemma 3 (Where A sits). Uniformly in m and eventually in n ,

$$F_{m-1}(n) \leq A(m, n) \leq F_{m+1}(n), \quad (4)$$

with the convention $F_k = F_0$ for $k \leq 0$.

Proof sketch. **Lower bound** ($A(m, n) \geq F_{m-1}(n)$, eventual in n , uniform in m): by induction on m . $m = 1$: $A(1, n) = n + 2 \geq n + 1 = F_0(n)$. Step: $A(m+1, n+1) = A(m, A(m+1, n)) \geq A(m, F_m(n)) \geq F_{m-1}(F_m(n))$ (inductive hypotheses), and $F_{m-1}(F_m(n)) \geq F_m(n+1)$ for n large (since $F_m(n) \geq n+1$ and F_{m-1} is increasing, one application of F_{m-1} to a value $\geq F_m(n)$ dominates the $(n+1)$ -fold iterate of F_{m-1} from $n+1$). **Upper bound** ($A(m, n) \leq F_{m+1}(n)$, eventual): by induction on m , $m = 0$ immediate; the step uses $A(m+1, n+1) = A(m, A(m+1, n)) \leq F_{m+1}(A(m+1, n))$ and the level-additivity of Lemma 2 to absorb the argument. The offsets $(-1, +1)$ are a matter of the convention (3); what is invariant is that A is a **bounded level-shift** from F_m — it runs *through* the hierarchy but never a level further ahead. $\square$

6. A is not primitive recursive

We are now ready for the main theorem.

Theorem 2. The Ackermann function $A : \mathbb{N}^2 \rightarrow \mathbb{N}$ is total and Turing-computable, but is **not** primitive recursive.

Proof. Totality and Turing-computability are §2.2 and §3. For non-PR-ness, consider the **diagonal** $a(n) = A(n, n)$.

Assume, for contradiction, that A is PR. Then $a(n) = A(n, n)$ is PR (a diagonal of a PR function is PR — it is the composition $n \mapsto A(n, n)$). By Lemma 2 (with $k = 1$, so the sum-size is just n), there exist m and N such that

$$a(n) = A(n, n) \leq F_m(n) \quad \text{for all } n \geq N. \quad (5)$$

On the other hand, Lemma 3's lower bound gives $A(n, n) \geq F_{n-1}(n)$ eventually in n , uniformly. Since the hierarchy is strictly increasing in its level — $F_{k+1}(n) > F_k(n)$ for

$n \geq 2$ — we have $F_{n-1}(n) > F_m(n)$ whenever $n - 1 \geq m + 1$, i.e. $n \geq m + 2$. Hence for all $n \geq \max(N, m + 2)$,

$$a(n) = A(n, n) \geq F_{n-1}(n) > F_m(n), \quad (6)$$

contradicting (5). $\square$

Corollary 2 (resource form). No Turing machine computing A has a primitive-recursive bound on its running time.

Proof. Immediate from Theorem 2 and Corollary 1. $\square$

This is the precise sense in which the Ackermann function “outruns” the primitive-recursive world: it is not that some particular machine is slow, but that **the function itself** cannot be certified by any PR clock. The diagonal $a(n) = A(n, n)$ is the object that climbs the fast-growing hierarchy without bound, and no single level F_m can bound it.

7. Proof-theoretic significance: the threshold of Peano Arithmetic

The primitive-recursive/total-computable divide has a sharp **proof-theoretic** shadow, which is where the relationship to formal systems — and, indirectly, to the halting problem — becomes visible.

7.1 Provable totality

- Every **primitive-recursive** function is provably total in very weak arithmetic (already in fragments with only Δ_0 or Σ_1 induction): its totality is witnessed by a straightforward induction mirroring its finite construction.
- A is **provably total in Peano Arithmetic (PA)**. The induction on m of §2.2, with the nested recursion handled by induction on n , is formalizable in PA.
- The **provably-total functions of PA** are exactly those dominated by the fast-growing hierarchy up to the ordinal ε_0 (the proof-theoretic ordinal of PA) — the classical ordinal-analysis result (Girard; see Hájek & Pudlák). In particular, a function f is provably total in PA iff $f(n) \leq F_\alpha(n)$ for some $\alpha < \varepsilon_0$.

Since A 's diagonal lives at a *finite* level of the hierarchy (F_{n-1} , far below F_{ε_0}), it sits comfortably inside PA's provable totality — but **above** the fragment that proves totality of all primitive-recursive functions. A is thus the threshold object: provably total in PA, yet not in the PR class.

7.2 The Kirby-Paris hydra interpretation

The most vivid modern interpretation of A 's growth is the **hydra game** (Kirby & Paris, 1982). A “hydra” is a finite rooted tree; a “move” consists of the hero cutting off a head, after which the hydra regrows a bounded number of copies of the stump. The game always terminates — the hydra is finite and each move decreases an associated ordinal $< \varepsilon_0$ — but the number of moves to kill a hydra of size n grows at the rate of the fast-growing hierarchy, hence like $A(n, n)$. Crucially, the statement “the hydra game always terminates” is **provable in PA but not in the weaker Δ_0 -induction fragment**. This gives a *non-arithmetic*, combinatorial reason why A grows as fast as it does and why its totality lands exactly at PA's threshold: A is, up to a level shift, the function measuring the termination time of a process whose well-foundedness is provable in PA but not below it.

7.3 Relation to the halting problem

The general problem “does this Turing machine halt on **all** inputs?” (i.e. “does it compute a total function?”) is **undecidable** — it is Π_2^0 -complete, strictly harder than the halting problem. The Ackermann function is the *concrete, positive* counterpart to this undecidability: it is a specific function whose totality we can *prove* (in PA) and whose machine M_A we

can write down, yet which lies beyond the primitive-recursive, PR-bounded-computation realm. A is thus the boundary between the functions a weak theory can certify as total (the PR functions) and those that are total but require the full induction of PA — the same induction strength that separates PA from its Δ_0 fragment in the hydra theorem.

8. Conclusion

The relationship between the Ackermann function and the Turing machine is not that the machine computes the function — that is shared by every total computable function. It is that A is the canonical, minimal witness to the strictness of the inclusion

$$\text{primitive recursive} \subsetneq \text{total Turing-computable},$$

and it realizes that strictness in three equivalent, precisely-stated ways:

1. **Functionally:** A is total and Turing-computable (§3) but not primitive recursive (§6), because its diagonal outruns every fixed level of the fast-growing hierarchy that exhausts the PR functions (Lemmas 2–3).
2. **Resource-wise:** by Theorem 1 and Corollary 2, A is Turing-computable but **no Turing machine computing it has a primitive-recursive time bound**. A is the first total function a machine can evaluate that no PR clock can certify.
3. **Proof-theoretically:** A is provably total in PA and measures the termination of the hydra game; its growth is a finite level of the hierarchy up to ε_0 that constitutes exactly the provable-totality strength of PA.

Each increment of A 's first argument adds one iteration level — successor, addition, multiplication, exponentiation, tetration — and it is precisely this unbounded ascent through the fast-growing hierarchy, still *within* it, that places A on the exact boundary between what a Turing machine can compute with provably bounded effort and what it can compute at all.

References

- **Ackermann, J.** (1928). *Zum Hilbertschen Aufbau der reellen Zahlen*. Mathematische Annalen **107**, 459–488.
- **Péter, R.** (1956–58). *Ein kombinatorischer Satz über die Funktionenklasse der primitiv-rekursiven Funktionen*. Acta Physico-Mathematica **6**, 1–10; see also *Die Theorie der rekursiven Funktionen* (Akadémiai Kiadó, 1956).
- **Goodstein, R. L.** (1947). *On the computable functions*. Journal of Symbolic Logic **12**, 127–131.
- **Kleene, S. C.** (1936). *General form of recursive function*. American Journal of Mathematics **57**, 254–263.
- **Church, A.** (1936). *An unsolvability problem in arithmetic*. Journal of the London Mathematical Society **11**, 218–228.
- **Turing, A. M.** (1936). *On computable numbers, with an application to the Entscheidungsproblem*. Proceedings of the London Mathematical Society **42**, 230–265.
- **Rogers, H.** (1967). *Theory of Recursive Functions and Effective Computability*. MIT Press.
- **Cutland, N. J.** (1980). *Computability: An Introduction to Recursive Function Theory*. Cambridge University Press.
- **Hájek, P., & Pudlák, P.** (1993). *Metamathematics of First-Order Arithmetic*. Cambridge University Press. (Provably-total functions of PA; ordinal ε_0 .)
- **Kirby, L., & Paris, J.** (1982). *Optimal incompleteness results for Peano Arithmetic*. In *Mathematical Logic: Oxford, 1985* (Lecture Notes in Math. **1305**), Springer. (The hydra game.)

- **Odifreddi, P. E.** (1992). *Classical Recursion Theory*. North-Holland.